

solids

Manuel complet

Géométrie 3D et figures de collège inspirées d'Asymptote

Typst 0.15.1 · version 0.3.0 · nibart 0.3.0

Projection 3D vectorielle, solides pédagogiques, surfaces paramétriques et annotations RTL

Ce manuel accompagne le package `@preview/solids3d:0.4.0`. Il décrit l'API publique et les conventions de rendu.

Sommaire

Sommaire	2
Présentation	4
Installation	4
Premier document	4
Exemples complets : code et résultat	5
Sphère, squelette et repère	5
Pavé droit annoté et diagonale réelle	5
Surface paramétrique et maillage	5
Courbe paramétrique avec la façade graph3	6
Polyèdres de collège supplémentaires	6
RTL : textes arabes, géométrie latine	6
Le modèle mental	7
Triplets, chemins et surfaces	7
Transformations	8
Projections et mise en page	8
Projection orthographique	8
Projection perspective	8
picture	9
Limites et axes	10
Chemins 3D	10
Lignes et splines	10
Arcs et cercles	11
Graphes 3D	11
Solides et surfaces	12
Solides de révolution	12
Tubes autour des chemins	13
Solides unitaires	13
Extrusion et cône sur une base	14
Pavé droit et boîte filaire	14
Dessin, squelettes et silhouettes	14
draw sur une surface	14
Squelette d'une révolution	15
Silhouette	15
Points, labels et flèches	16
Éclairage, matériaux et transparence	16
Couleurs et opacité	16
Matériaux	17
Lumières	17
Options d'ombrage	17
Styles vectoriels optionnels : vintage et pencil draw	18
Planche vintage : papier, encre et hachures	18
Pencil draw : trait irrégulier et ombrage par trame	19
Contour normal et remplissage gravé	20
Sketch 2D statique et projection 3D	20
Surfaces paramétriques et maillages	21
Grille paramétrique	21
Champ de hauteurs	21
Maillage indexé	21
Figures pédagogiques collège	22
Sommets et flèches de rappel	22
Lignes de cote	22
Codages d'arêtes et d'angles	23
RTL et textes multilingues	23
Nouveautés 0.2 : nibart, gravure, nœuds, accolades et planches	23
Moteur de plume : engine	24

Le style engraving : la gravure au burin	24
Nœuds et entrelacs : knot3	25
Accolades calligraphiques : brace3	26
Cartouche gravé : plate	27
Correspondance avec Asymptote	27
Commandes et paramètres — code à gauche, rendu à droite	28
Construire une figure : picture, projection et draw	28
Chemins et courbes 3D	28
Solides, surfaces paramétriques et tubes	29
Annotations pédagogiques	30
Référence API condensée	30
Projections	30
Chemins	30
Solides et surfaces	30
Rendu	31
Éducation	31
Façades de modules Asymptote	31
Performance et qualité	31
Limites connues	32
Dépannage	32
Typst ne trouve pas le package	32
Les textes arabes sont mal placés	32
Une diagonale n'apparaît pas	32
Les coutures des patches sont visibles	32
La figure est trop lente	32
Organisation et livraison	32
Licence et références	33

Présentation

`solids3d` est une réimplémentation Typst indépendante pour produire des figures de géométrie dans l'espace directement dans un document Typst. Son style `pencil` calcule les enveloppes de plume avec `nibart` ; le reste du rendu 3D et de l'interface est propre au paquet. Elle reprend les conventions publiques et les idées géométriques de `solids`, `three`, `three_surface`, `three_light`, `graph3` et `obj` d'Asymptote, mais le rendu final est une composition 2D vectorielle après projection.

Le package sait représenter :

- les solides de révolution : sphères, cônes, cylindres, tores et profils quelconques ;
- les solides usuels : cube, pavé droit, pyramide, prisme, disque, plan, frustum et hémisphère ;
- les chemins 3D, courbes de Bézier, arcs, graphes et transformations ;
- les surfaces paramétriques, champs de hauteurs et maillages indexés ;
- les squelettes visibles/cachés, silhouettes, axes, flèches, points, dimensions et légendes ;
- les textes multilingues, y compris l'arabe et l'hébreu en RTL.
- (0.2) la gravure au burin (`style: "engraving"`), les nœuds et entrelacs tubulaires (`knot3`), les accolades calligraphiques (`brace3`) et le cartouche `plate`, tous dessinés avec `nibart`.

Idée importante. Une figure `picture(...)` est une boîte Typst ordinaire. Elle peut être placée dans une figure, une grid, une colonne, un tableau ou un document pédagogique sans exporter d'image intermédiaire.

Installation

Depuis l'archive de livraison :

```
unzip solids3d-0.4.0.zip
cd solids3d-0.4.0
./install.sh
```

Dépendance. Depuis la 0.2, le package importe `@preview/nibart:0.3.0` (Typst le télécharge automatiquement quand `nibart` est publié ; hors ligne, extrayez `nibart-0.3.0-universe.zip` dans le dossier des paquets Typst).

Dans un document :

```
#import "@preview/solids3d:0.4.0": *
```

Pour travailler sans installer le package, un document situé dans le dépôt peut utiliser :

```
#import "lib.typ": *
```

Le script `make-release.sh` installe le package, compile les tests et tous les exemples publics, puis produit l'archive et sa somme SHA-256.

Premier document

```
#import "@preview/solids3d:0.4.0": *
#set page(width: auto, height: auto, margin: 1cm)

#let P = orthographic(5, 4, 3)
#let b = sphere(0, 1)

#picture(size: 6cm, projection: P,
  draw(surface(b), withopacity(paleblue, 0.55)),
  draw(b, m: 5, frontpen: blue,
    backpen: pen(blue, linetype("8 8")),
    longitudinalpen: nullpen),
  xaxis3($x$, Arrow3),
  yaxis3($y$, Arrow3),
  zaxis3($z$, Arrow3),
)
```

Les commandes `draw(...)` renvoient des commandes de dessin. Elles sont évaluées par `picture`, qui connaît la projection, la lumière, l'échelle et les options de sortie.

Exemples complets : code et résultat

Les exemples suivants sont autonomes. Dans chaque panneau, le code de gauche peut être copié dans un document Typst utilisant Universe ; le panneau de droite est produit par le même code et reste entièrement vectoriel.

Sphère, squelette et repère

Exemple 1 — surface, lignes visibles et cachées

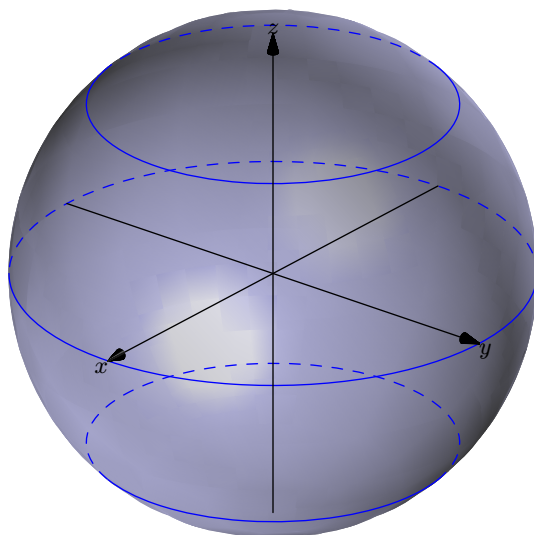
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let P = orthographic(5, 4, 3)
#let b = sphere(0, 1)

#picture(width: 7cm, projection: P,
  draw(surface(b), withopacity(paleblue, 0.55)),
  draw(b, m: 5, frontpen: blue,
    backpen: pen(blue, dashed), longitudinalpen:
    nullpen),
  xaxis3($x$, Arrow3),
  yaxis3($y$, Arrow3),
  zaxis3($z$, Arrow3),
)
```

Résultat vectoriel



Pavé droit annoté et diagonale réelle

Cet exemple montre la convention collège recommandée : les sommets restent latins, les cotes utilisent cm, et la diagonale est dessinée avant d'être éventuellement désignée par une flèche.

Exemple 2 — solide de collège avec cotes

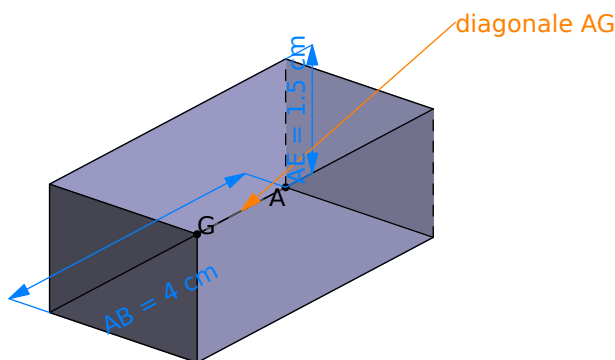
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let v = cuboid-vertices(length: 4, width: 2,
  height: 1.5)
#let P = orthographic(5, 4, 3)

#picture(width: 8cm, projection: P,
  draw(cuboid(length: 4, width: 2, height: 1.5),
    withopacity(paleblue, 0.65)),
  ..cuboid-edges(v),
  diagonal3(v.A, v.G, pen: pen(gray(0.35), 0.6pt,
    dashed)),
  point3([A], v.A, align: SW),
  point3([G], v.G, align: NE),
  dimension3([AB = 4 cm], v.A, v.B,
    offset: (0, -0.55, 0), align: S, pen:
    royalblue),
  dimension3([AE = 1.5 cm], v.A, v.E,
    offset: (-0.45, 0, 0), align: W, pen:
    royalblue),
  callout3([diagonale AG], interp(v.A, v.G, 0.5),
    (-1, 1.5, 2), pen: orange),
)
```

Résultat vectoriel



Surface paramétrique et maillage

Exemple 3 — surface paramétrique et maillage

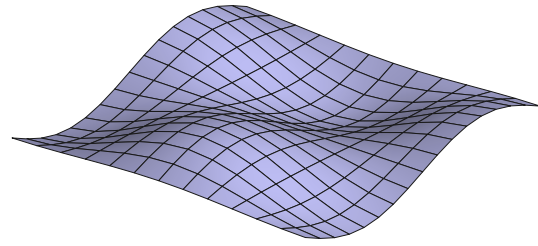
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let P = orthographic(5, 4, 3)
#let wave = surface-grid(
  (u, v) => (u, v, 0.22 * calc.sin(2 * u) *
    calc.cos(2 * v)),
  -1.5, 1.5, -1.5, 1.5,
  nu: 16, nv: 12,
)

#picture(width: 7cm, projection: P,
  draw(wave, withopacity(paleblue, 0.9), meshpen:
    pen(black, 0.25pt)),
)
```

Résultat vectoriel



Courbe paramétrique avec la façade graph3

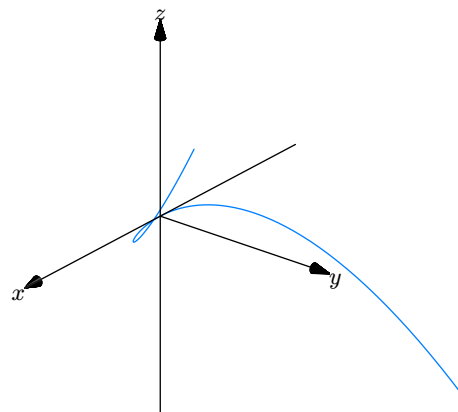
Exemple 4 — module graph3.typ

Code Typst

```
#import "@preview/solids3d:0.4.0": graph3-plot

#graph3-plot(
  t => (t, t * t, t * t * t),
  -1, 1,
  n: 80,
  pen: royalblue,
  axes: true,
)
```

Résultat vectoriel



Polyèdres de collège supplémentaires

Exemple 5 — tétraèdre et octaèdre réguliers

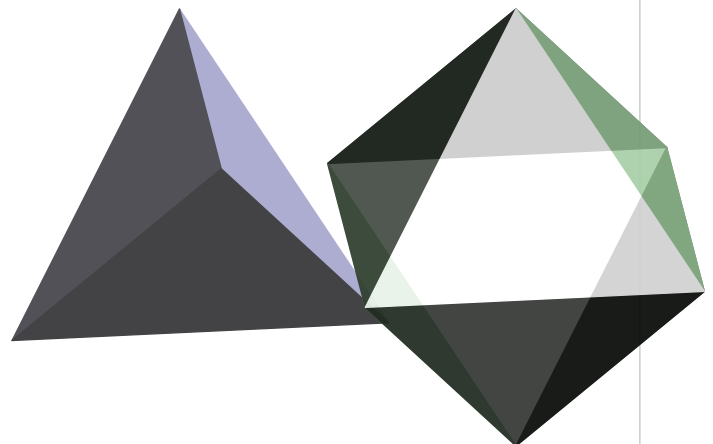
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let P = orthographic(5, 4, 3)
#let t = regular-tetrahedron(radius: 1)
#let o = regular-octahedron(radius: 1)

#grid(columns: 2, gutter: 8pt,
  picture(width: 5cm, projection: P,
    draw(t, withopacity(paleblue, 0.75)),
  ),
  picture(width: 5cm, projection: P,
    draw(o, withopacity(palegreen, 0.75)),
  ),
)
```

Résultat vectoriel



RTL : textes arabes, géométrie latine

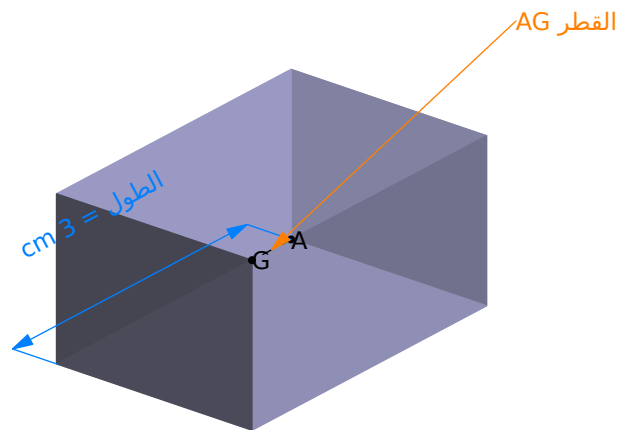
Exemple 6 — RTL sans traduire les symboles géométriques

Code Typst

```
#import "@preview/solids3d:0.4.0": *
#set text(font: "DejaVu Sans", lang: "ar", dir: rtl)

#let v = cuboid-vertices(length: 3, width: 2,
height: 1.5)
#picture(width: 8cm, projection: orthographic(5, 4,
3),
draw(cuboid(length: 3, width: 2, height: 1.5),
withopacity(paleblue, 0.65)),
diagonal3(v.A, v.G),
point3([A], v.A), point3([G], v.G),
dimension3([3 = الطول cm], v.A, v.B,
offset: (0, -0.45, 0), pen: royalblue),
callout3([القطر AG], interp(v.A, v.G, 0.5),
(-1, 1.5, 2), pen: orange),
)
```

Résultat vectoriel



Ces panneaux utilisent @preview/solids3d:0.4.0 dans le code montré à gauche. Pour compiler le manuel depuis le dépôt, son import de travail est relatif (lib.typ) ; le code de l'utilisateur, lui, doit utiliser l'import Universe.

Le modèle mental

Triplets, chemins et surfaces

Les points et vecteurs sont des triplets numériques. Le panneau suivant montre à la fois le calcul des vecteurs et les chemins qui en résultent :

Triplets, lignes, splines et cercles

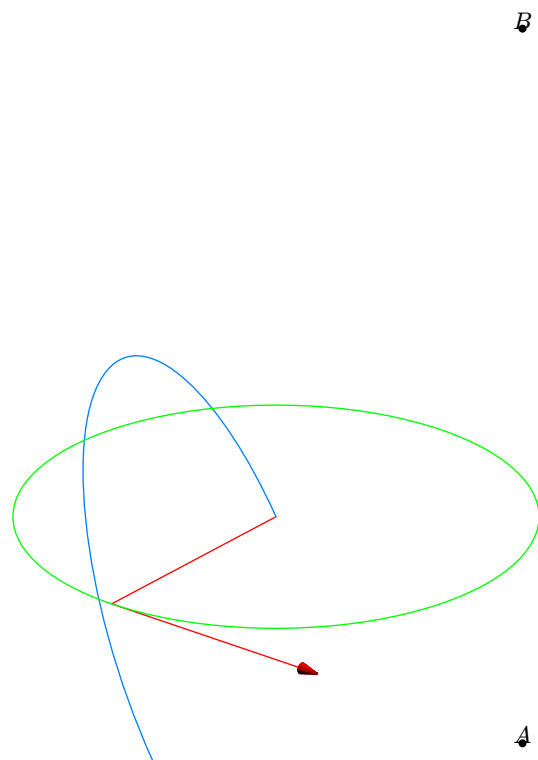
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let A = (1, 2, 0)
#let B = (1, 2, 3)
#let v = vsub(B, A)
#let longueur = vabs(v)
#let segment = line3(0, X, (1, 1, 0))
#let courbe = spline3(0, (1, 0, 1), (2, 1, 0))
#let cercle = Circle3(0, 1, normal: Z, n: 24)

#picture(width: 7cm, projection: orthographic(5, 4,
3),
draw(segment, red, Arrow3),
draw(courbe, royalblue),
draw(cercle, green),
dot(Label([A$]), align: N), A),
dot(Label([B$]), align: N), B),
)
```

Résultat vectoriel



Les constantes de base sont $0 = (0, 0, 0)$, X , Y et Z . Les opérations usuelles sont `vadd`, `vsub`, `vmul`, `vneg`, `vdot`, `vcross`, `vabs`, `vunit`, `vangle` et `vproject`.

Une surface est une collection de patches. Elle peut provenir d'un chemin plan, d'un solide de révolution, d'une extrusion, d'un maillage ou d'une union de surfaces.

Transformations

Les transformations sont des matrices 4×4 . `apply(T, objet)` est la forme Typst du produit $T * \text{objet}$ d'Asymptote :

Transformation d'un cube

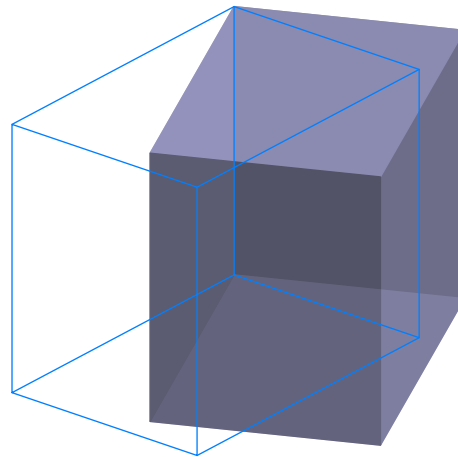
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let T = compose(
  shift(1, 0, 0),
  rotate3(25, Z),
  scale3(1.2, y: 0.8, z: 1),
)
#let objet = apply(T, unitcube)

#picture(width: 6cm, projection: orthographic(5, 4, 3),
  draw(objet, withopacity(paleblue, 0.7)),
  draw(box3((1, 0, 0), (2.2, 0.8, 1)), royalblue),
)
```

Résultat vectoriel



Fonctions disponibles : `identity4`, `shift`, `scale3`, `xscale3`, `yscale3`, `zscale3`, `rotate3`, `reflect3`, `align3`, `compose`, `tpoint`, `tvector`, `tmul` et `inverse4`.

`apply(T, objet)` est volontairement utilisé à la place de $T * \text{objet}$. C'est la convention du package et elle fonctionne pour les points, chemins, surfaces, révolutions et tableaux d'objets.

Projections et mise en page

Projection orthographique

Deux réglages de projection orthographique

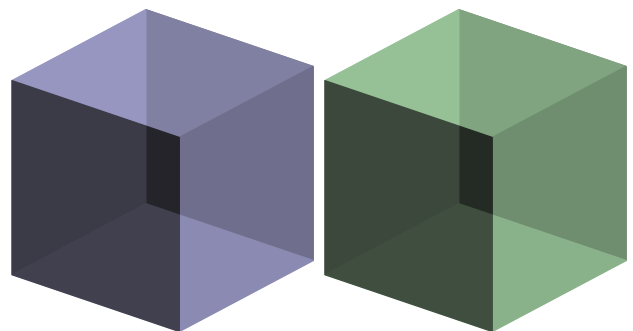
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let P = orthographic(5, 4, 3)
#let P2 = orthographic(
  camera: (5, 4, 3),
  up: Z,
  target: 0,
  zoom: 0.95,
)

#grid(columns: 2, gutter: 6pt,
  picture(width: 4cm, projection: P,
    draw(unitcube, withopacity(paleblue, 0.7))),
  picture(width: 4cm, projection: P2,
    draw(unitcube, withopacity(palegreen, 0.7))),
)
```

Résultat vectoriel



`orthographic(5, 4, 3)` regarde l'origine depuis la caméra (5,4,3) sans réduire les objets éloignés. Les vues prédéfinies sont `LeftView`, `RightView`, `FrontView`, `BackView`, `BottomView` et `TopView`.

Projection perspective

Deux projections en perspective

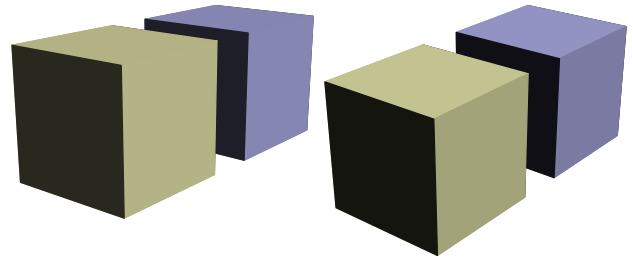
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let P = perspective(5, 4, 2)
#let Q = perspective(
  camera: (5, 4, 3), up: Z, target: 0, zoom: 1,
)

#grid(columns: 2, gutter: 6pt,
  picture(width: 4cm, projection: P,
    draw(apply(shift(-0.7, 0, 0), unitcube),
      paleblue),
    draw(apply(shift(0.7, 0, 0), unitcube),
      paleyellow)),
  picture(width: 4cm, projection: Q,
    draw(apply(shift(-0.7, 0, 0), unitcube),
      paleblue),
    draw(apply(shift(0.7, 0, 0), unitcube),
      paleyellow)),
  )
```

Résultat vectoriel



La perspective conserve le point de fuite et réduit les objets éloignés. Les courbes sont interpolées davantage en perspective afin d'éviter les erreurs de projection des arcs.

picture

Signature principale :

```
picture(
  ...commands,
  size: auto,
  width: auto,
  height: auto,
  unitsize: 1cm,
  projection: currentprojection,
  light: currentlight,
  limits: none,
  margin: 0pt,
  inset: 0pt,
  fill: none,
  stroke: none,
  radius: 0pt,
  clip: false,
  baseline: 0pt,
  zsort: "command",
  shading: "smooth",
  refine: true,
  backend: "svg",
)
```

size ajuste la figure dans une boîte carrée ou rectangulaire. width et height imposent une dimension. margin et inset ajoutent de l'espace autour de la géométrie et des labels. Les labels sont pris en compte dans la boîte finale.

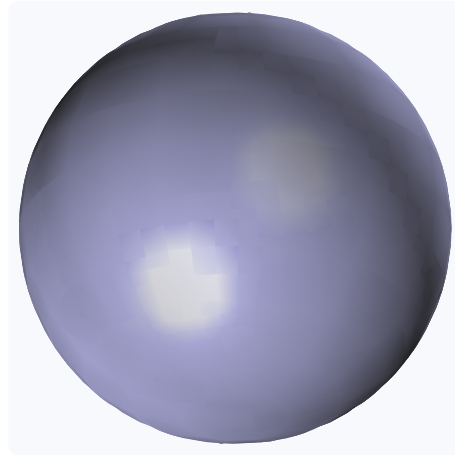
Boîte picture avec fond, marge et arrondi

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#picture(
  width: 6cm,
  projection: orthographic(5, 4, 3),
  fill: rgb("f7f9fc"),
  radius: 4pt,
  inset: 4pt,
  draw(surface(sphere(0, 1)), withopacity(paleblue,
0.65)),
)
```

Résultat vectoriel



Limites et axes

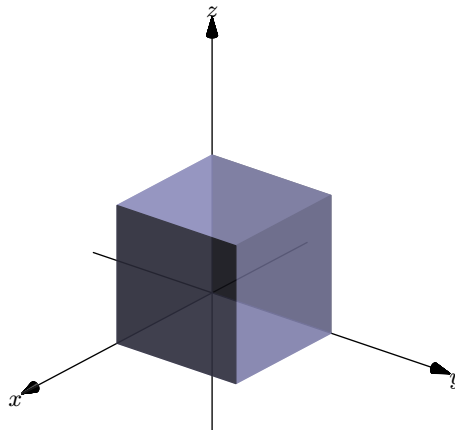
Repère 3D et limites explicites

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#picture(size: 6cm, projection: orthographic(5, 4,
3),
  limits((-1, -1, -1), (2, 2, 2)),
  xaxis3($x$, Arrow3), yaxis3($y$, Arrow3),
  zaxis3($z$, Arrow3),
  draw(unitcube, withopacity(paleblue, 0.7)),
)
```

Résultat vectoriel



`limits(min, max)` fournit les bornes par défaut aux axes. Un axe peut être contrôlé avec `min:` et `max:`. `axes3` construit les trois axes en une seule commande.

Chemins 3D

Lignes et splines

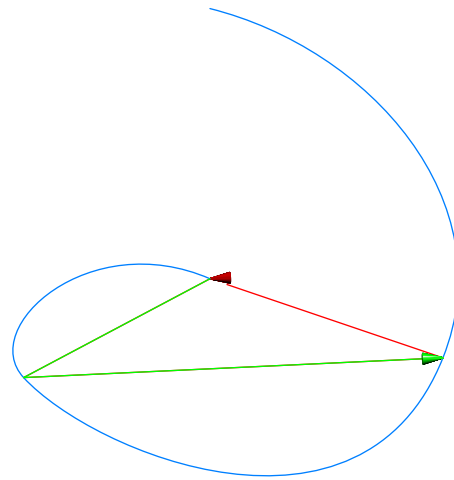
Chemins polygonaux, Hobby 3D et guide

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let a = line3(0, X, Y, cyclic: true)
#let b = spline3(0, X, Y, Z)
#let c = guide3(
  0, (out: X), "...", X,
  ("in": Y), "--", Y,
)
#picture(width: 7cm, projection: orthographic(5, 4,
3),
  draw(a, red, Arrow3),
  draw(b, royalblue),
  draw(c, green, Arrow3),
)
```

Résultat vectoriel



Les connecteurs "--" produisent des segments droits. Les connecteurs "." utilisent une version 3D de l'algorithme de Hobby. Les fonctions point, dir, precontrol, postcontrol, segment, subpath, reverse, join, arclength, arctime et reltime inspectent ou découpent les chemins.

Arcs et cercles

Arcs et cercles dans différents plans

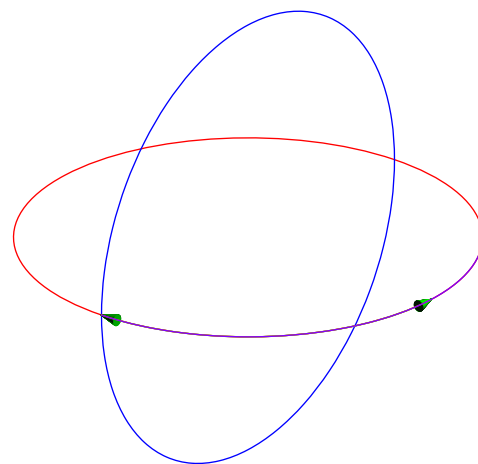
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let cercle1 = circle3(0, 1, normal: Z)
#let cercle2 = Circle3(0, 1, normal: Y, n: 32)
#let arc = arc3(0, X, Y, normal: Z)
#let arc2 = Arc3(0, X, Y, normal: Z, n: 8)
#let arc3d = Arc3-angles(0, 1, 30, 0, 70, 120,
normal: Z)

#picture(width: 7cm, projection: orthographic(5, 4,
3),
  draw(cercle1, red), draw(cercle2, blue),
  draw(arc, green, Arrows3), draw(arc2, orange),
  draw(arc3d, purple),
)
```

Résultat vectoriel



Les majuscules Circle3 et Arc3 permettent de choisir le nombre de segments. Les minuscules correspondent aux constructions compactes usuelles.

Graphes 3D

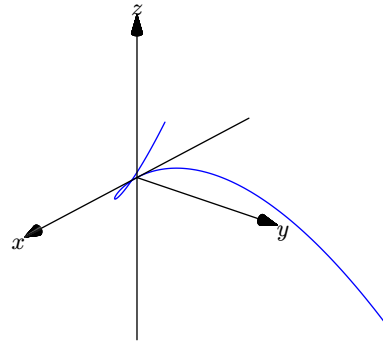
Graphe d'une courbe paramétrique 3D

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let g = graph3(
  t => (t, t * t, t * t * t),
  -1, 1, n: 80, join: "..",
)
#picture(size: 5cm, projection: orthographic(5, 4, 3),
  draw(g, blue), xaxis3($x$, Arrow3), yaxis3($y$, Arrow3), zaxis3($z$, Arrow3),
)
```

Résultat vectoriel



Solides et surfaces

Solides de révolution

Un profil est tourné autour d'une droite :

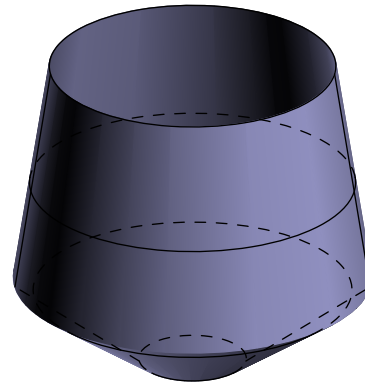
Surface de révolution à partir d'un profil

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let profil = line3(
  (0.3, 0, 0), (1.0, 0, 0.5), (0.8, 0, 1.8),
)
#let r = revolution(profil, Z)
#picture(size: 5cm, projection: orthographic(5, 4, 3),
  draw(surface(r), paleblue),
  draw(r, m: 4, frontpen: black, backpen: dashed),
)
```

Résultat vectoriel



Les constructeurs usuels sont :

Sphère, cylindre, cône et tore

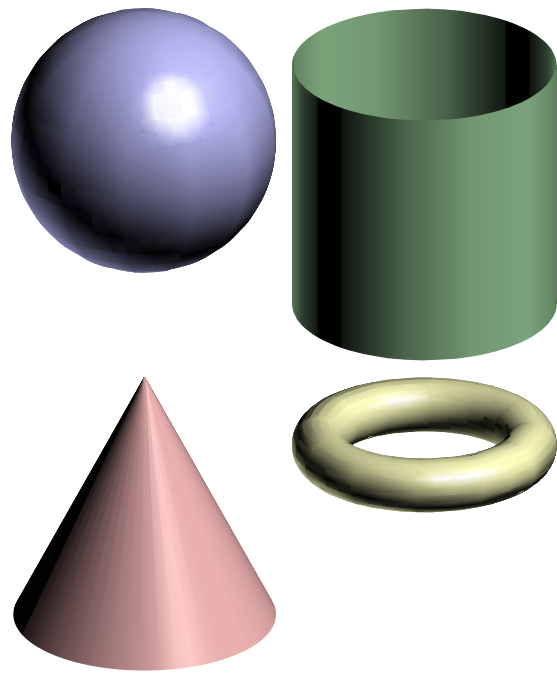
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let b = sphere(0, 1, n: 16)
#let c = cylinder(0, 1, 2, Z)
#let k = cone(0, 1, 2, Z, 1)
#let t = torus(2, 0.5, n: 20)

#grid(columns: 2, gutter: 6pt,
  picture(width: 3.5cm, projection: orthographic(5,
4, 3), draw(surface(b), paleblue)),
  picture(width: 3.5cm, projection: orthographic(5,
4, 3), draw(surface(c), palegreen)),
  picture(width: 3.5cm, projection: orthographic(5,
4, 3), draw(surface(k), palered)),
  picture(width: 3.5cm, projection: orthographic(6,
4, 3), draw(surface(t), paleyellow)),
)
```

Résultat vectoriel



`surface(r, n: 24)` transforme une révolution en surface maillée. `angle1` et `angle2` permettent de ne tourner qu'une portion du profil.

Tubes autour des chemins

`tube3` (alias `tube`) épaissit un `path3` en une surface polygonale. Il est adapté aux hélices, fils et tiges et reste entièrement autonome : `width` est le diamètre, `n` le nombre de côtés, `samples` raffine les segments de Bézier et `caps: true` ferme les extrémités. Le repère normal est transporté le long du chemin afin d'éviter les retournements brutaux d'un repère de Frenet près des points d'inflexion.

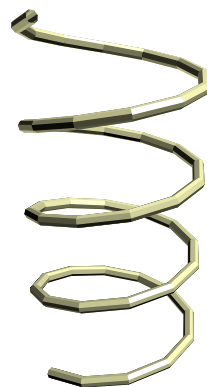
Tube polygonal autour d'une hélice

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let tau = 2 * calc.pi
#let helix = graph3(
  t => (1.0 * calc.cos(t * 1rad), 1.0 * calc.sin(t *
1rad), -1 + t / tau),
  0, 4 * tau, n: 48, join: "--",
)
#let wire = tube3(helix, width: 0.12, n: 8)
#picture(size: 5cm, projection: perspective(4, 5,
3),
  draw(surface(wire), paleyellow),
)
```

Résultat vectoriel



Solides unitaires

Le package fournit `unitsphere`, `unithemisphere`, `unitcylinder`, `unitcone`, `unitsolidcone`, `unitcube`, `unitdisk`, `unitplane`, `unitfrustum`, `unitbox`, `unitsquare3` et `unitcircle3`.

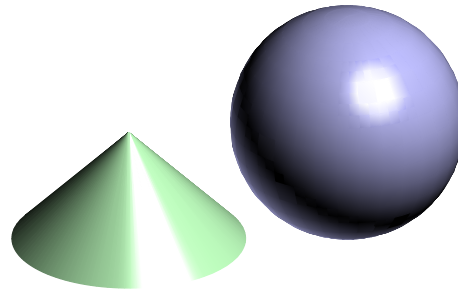
Deux solides unitaires transformés

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let scène = (
  draw(apply(shift(-1.5, 0, 0), unitsphere),
    paleblue),
  draw(apply(shift(1.5, 0, 0), unitcone),
    palegreen),
)
#picture(size: 6cm, projection: orthographic(5, 4,
3), ..scène)
```

Résultat vectoriel



unitbox est un tableau d'arêtes. unitcube est une surface fermée. Cette différence est utile pour choisir entre fil de fer et faces remplies.

Extrusion et cône sur une base

Extrusion et cône sur une base hexagonale

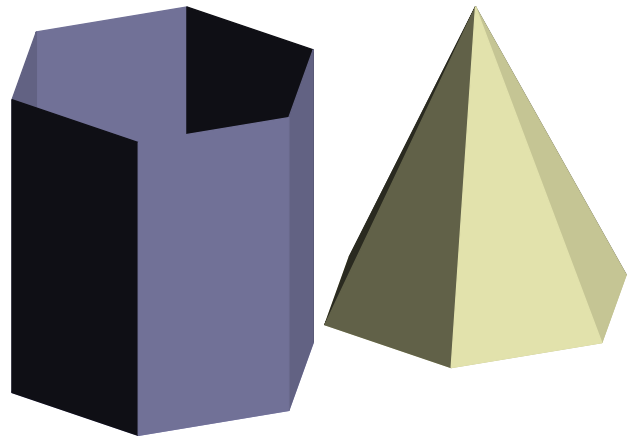
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let base = regular-polygon3(center: 0, radius: 1,
n: 6, angle: 30)
#let prisme = surface(extrude(base, (0, 0, 2)),
surface(base))
#let pyramide = surface(cone-over(base, (0, 0, 2)),
surface(base))

#grid(columns: 2, gutter: 6pt,
picture(width: 4cm, projection: orthographic(5, 4,
3), draw(prisme, paleblue)),
picture(width: 4cm, projection: orthographic(5, 4,
3), draw(pyramide, paleyellow)),
)
```

Résultat vectoriel



extrude(path, vecteur, n: ...) construit les faces latérales. cone-over relie chaque segment de la base à un sommet.

Pavé droit et boîte filaire

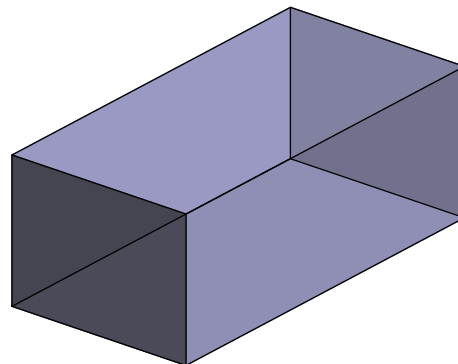
Pavé droit plein et boîte filaire

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let pavé = cuboid(length: 4, width: 2, height: 1.5)
#let arêtes = box3(0, (4, 2, 1.5))
#picture(size: 6cm, projection: orthographic(5, 4,
3),
draw(pavé, withopacity(paleblue, 0.65)),
draw(arêtes, black),
)
```

Résultat vectoriel



cuboid et pave-droit appartiennent au module pédagogique. Pour un simple pavé filaire, box3 suffit.

Dessin, squelettes et silhouettes

draw sur une surface

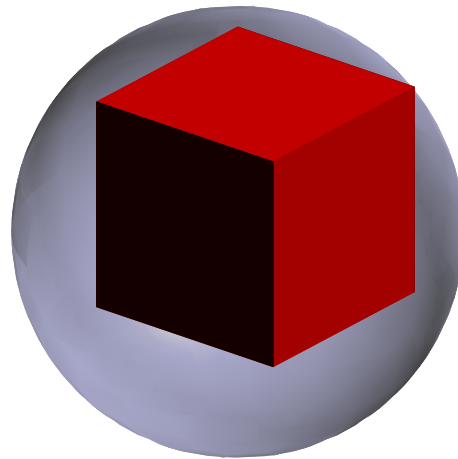
Surface translucide et matériau métallique

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#picture(size: 6cm, projection: orthographic(5, 4, 3),
  draw(surface(sphere(0, 1)), withopacity(paleblue, 0.5)),
  draw(surface(unitcube), material(
    diffuse: red, specular: white, shininess: 0.8,
  )),
)
```

Résultat vectoriel



La couleur ou le pen suivant l'objet devient le matériau diffus. Un deuxième pen sur une surface peut servir de meshpen :

```
draw(unitcube, withopacity(blue, 0.65), orange)
```

La surface est dessinée patch par patch, du plus éloigné au plus proche dans chaque commande `draw(surface(...))`.

Squelette d'une révolution

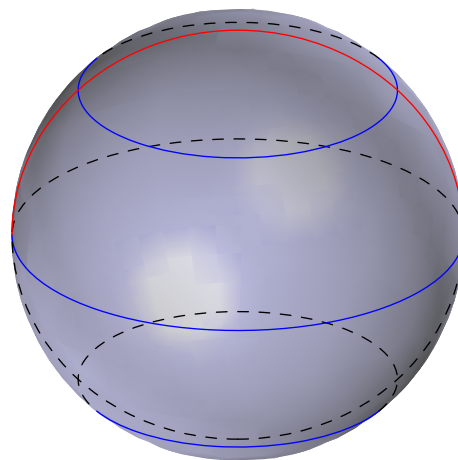
Squelette visible et caché d'une sphère

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let b = sphere(0, 1)
#let P = orthographic(5, 4, 3)
#let s = skeleton(b, m: 5, P: P)
#picture(size: 6cm, projection: P,
  draw(surface(b), withopacity(paleblue, 0.45)),
  draw(s.transverse.back, dashed),
  draw(s.transverse.front, blue),
  draw(s.longitudinal.back, dashed),
  draw(s.longitudinal.front, red),
)
```

Résultat vectoriel



`transverse(b, t, P)` renvoie `.front` et `.back`. `longitudinal(b, P)` renvoie également une paire de chemins. `skeleton` regroupe les deux familles. Les parties cachées ne sont pas masquées par un depth-buffer : elles sont produites séparément et il faut les dessiner avant les parties visibles.

Silhouette

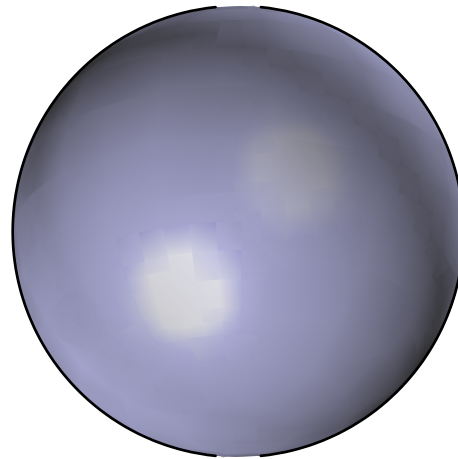
Silhouette apparente raffinée

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let b = sphere(0, 1)
#let P = orthographic(5, 4, 3)
#picture(size: 6cm, projection: P,
  draw(surface(b), withopacity(paleblue, 0.6)),
  draw(silhouette(b, m: 64, P: P), black + 1pt),
)
```

Résultat vectoriel



Pour les surfaces de révolution, `refine: true` découpe analytiquement les patches qui traversent la silhouette apparente. Cela réduit les festons du bord à faible nombre de tranches.

Points, labels et flèches

Labels, points et flèches 3D

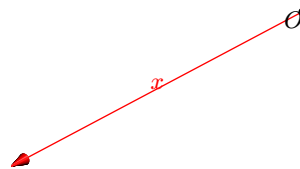
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#picture(size: 6cm, projection: orthographic(5, 4,
3),
  draw(Label($x$, align: N), line3(0, vmul(1.5, X)),
    red, Arrow3),
  dot(Label($O$, align: SW), 0, black),
  label([sommet], (0, 0, 1), align: N),
)
```

Résultat vectoriel

sommet



Les flèches disponibles sont `Arrow3`, `BeginArrow3`, `Arrows3`, `Arrow`, `BeginArrow` et `Arrows`. Les variantes 3 construisent une tête conique 3D ; les autres utilisent une tête plate projetée.

Éclairage, matériaux et transparence

Couleurs et opacité

Couleurs, opacité et opérations colorimétriques

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let c1 = rgbf(0.2, 0.5, 0.9)
#let c2 = withopacity(c1, 0.45)
#let c3 = color-add(paleblue, white)
#let c4 = color-scale(0.8, c1)

#grid(columns: 4, gutter: 4pt,
  circle(fill: c1, radius: 18pt),
  circle(fill: c2, radius: 18pt),
  circle(fill: c3, radius: 18pt),
  circle(fill: c4, radius: 18pt),
)
```

Résultat vectoriel



Les couleurs usuelles sont black, white, red, green, blue, cyan, magenta, yellow, les variantes pale..., light..., medium..., heavy..., deep..., ainsi que orange, purple, royalblue, brown et olive.

Matériaux

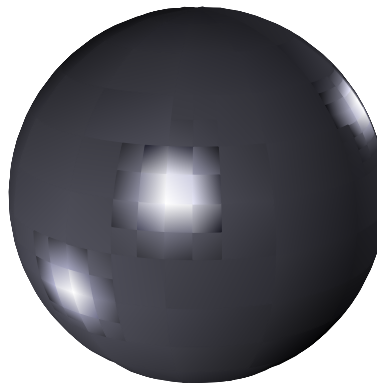
Matériau métallique éclairé

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let métal = material(
  diffuse: gray(0.25), specular: white, shininess:
0.95,
)
#picture(size: 5cm, light: White,
  draw(surface(sphere(0, 1)), métal),
)
```

Résultat vectoriel



Un matériau contient diffuse, emissive, specular, opacity et shininess. Les noms Asymptote diffusepen, emissivepen et specularpen sont également acceptés.

Lumières

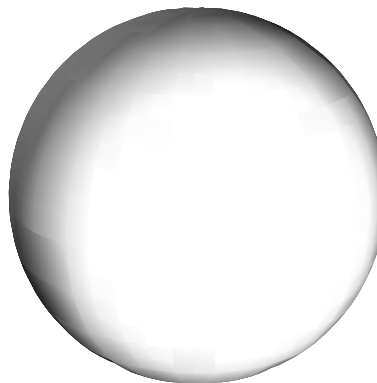
Lumière ponctuelle chaude

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#picture(size: 5cm,
  light: light(paleyellow, (5, -5, 10), (0, 0,
-10)),
  draw(surface(sphere(0, 1)), paleblue),
)
```

Résultat vectoriel



Les lumières prédéfinies sont Headlamp, Viewport, White et nolight. Une lumière peut être passée comme triplet, ce qui crée une lumière ponctuelle orientée dans le repère caméra.

Options d'ombrage

- shading: "smooth": couleur aux coins et au centre, gradient vectoriel et subdivision adaptative ;

- shading: "flat" : une couleur éclairée par patch, plus rapide et compacte ;
- shading: "none" : couleur diffuse directe, sans éclairage ;
- refine: false : désactive le raffinement de silhouette et la subdivision adaptative ;
- backend: "svg" : backend vectoriel SVG intégré au PDF ;
- backend: "typst" : courbes Typst natives, sans gradients SVG.

Pour une fiche pédagogique légère, commencez par shading: "flat" ou shading: "none". Pour une figure de publication, gardez les valeurs par défaut shading: "smooth" et refine: true.

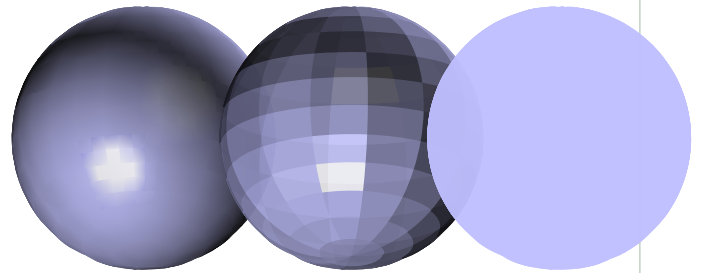
Comparaison des modes d'ombrage

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let scène = draw(surface(sphere(0, 1)),
  withopacity(paleblue, 0.8))
#grid(columns: 3, gutter: 5pt,
  picture(width: 3.5cm, shading: "smooth", scène),
  picture(width: 3.5cm, shading: "flat", scène),
  picture(width: 3.5cm, shading: "none", scène),
)
```

Résultat vectoriel



Styles vectoriels optionnels : vintage et pencil draw

Les effets visuels inspirés des planches scientifiques anciennes sont intégrés comme une couche optionnelle de solids3d. Ils ne remplacent ni la projection orthographique/perspective, ni le tri des patches, ni l'émission SVG/native : la sortie demeure composée de chemins vectoriels nets, déterministes et imprimables. Le mode pencil est volontairement monochrome : les aplats de surface implicites sont retirés et le papier reste visible ; la lumière est suggérée par la densité des hachures et du stippling, qui restent des couches séparées du contour. Une couleur explicite reste possible pour une plume ou un matériau particulier, mais elle devient alors un choix de l'auteur. Sans style, le rendu historique du package reste inchangé.

Depuis la 0.4, le paquet ne contient plus de module WebAssembly : les enveloppes de plume sont calculées par nibart, et une surface sans matériau, hatch: ni stipple: est gravée par le mécanisme du style engraving (trame de lignes dont l'épaisseur suit l'éclairage, lignes cachées éliminées) avec une trame plus légère, sépia en vintage, graphite ondulé en pencil ; l'option engraving: de picture règle cette trame. Un hatch:, un stipple: ou un matériau explicite garde le hachage par patch projeté.

Les contours implicites des solides sont des enveloppes remplies, pas des traits colorés répétés. Dans style: "pencil" ou "vintage", les lignes cachées issues du backpen implicite sont supprimées ; un backpen explicitement fourni reste une demande de l'auteur. Le stippling est une texture séparée et opt-in : ses points sont échantillonnés dans les patches projetés, avec une graine déterministe, puis émis comme cercles vectoriels indépendants du contour. Les hachures et les points peuvent ainsi suivre la lumière sans peindre un ton gris continu.

Planche vintage : papier, encre et hachures

style: "vintage" choisit un papier chaud, une encre sépia et des contours légèrement gravés. Les surfaces implicites sont gravées par la trame éclairée ; les backpen implicites sont retirés et un backpen explicite est conservé. vintage-hatch(...) explicite donne le hachage par patch projeté (spacing, angle, cross...) ; count, length, crosshatch et seed sont acceptés mais ignorés depuis la 0.4. vintage-stipple(...) applique le même principe à des points sur les sphères standards ; choisissez distribution: "random" ou distribution: "fibonacci" et utilisez hatch: none pour une trame de points distincte. count, min-size, max-size, gamma et seed contrôlent une trame reproductible, avec une densité et une taille ajustées par l'incidence de la lumière lorsque lit: true.

Vintage — hachures compatibles avec un solide 3D

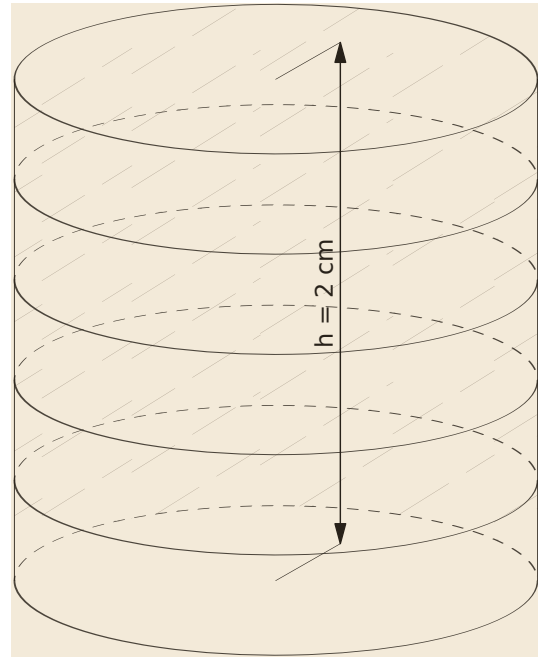
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let P = orthographic(6, 3, 2)
#let b = cylinder(0, 1, 2)

#picture(width: 7cm, projection: P, style:
"vintage",
  draw(surface(b), hatch: vintage-hatch(spacing:
0.22, angle: 32)),
  draw(b, m: 6,
    frontpen: pencil-pen(paint: vintage-ink,
thickness: 0.62pt),
    backpen: pencil-pen(paint: vintage-pale-ink,
thickness: 0.52pt),
    longitudinalpen: pencil-pen(paint: vintage-ink,
thickness: 0.38pt)),
  dimension3([h = 2 cm], (0, 0, 0), (0, 0, 2),
    offset: (-0.55, 0, 0), pen: vintage-ink),
)
```

Résultat vectoriel



Pencil draw : trait irrégulier et ombrage par trame

style: "pencil" applique une enveloppe de plume elliptique remplie et déterministe à toutes les plumes de chemins produites par draw ; il ne répète pas des traits colorés transparents. Les lignes cachées du squelette sont supprimées lorsqu'elles proviennent du backpen implicite ; pour étudier une arête cachée, passez explicitement un backpen. La surface reste sur le papier et ses ombres sont suggérées par des hachures croisées ou un stippling séparé, jamais par un aplat bleu ou violet. samples peut définir des axes elliptiques variables le long de l'arclength, tandis que pressure transmet le profil de la plume et provoque des ruptures lorsque la largeur devient imprimable ; une valeur numérique reste acceptée comme raccourci. roughness et seed sont conservés pour compatibilité de l'API ; l'enveloppe est calculée par nibart.

Pencil draw — trait vectoriel et ombrage par trame

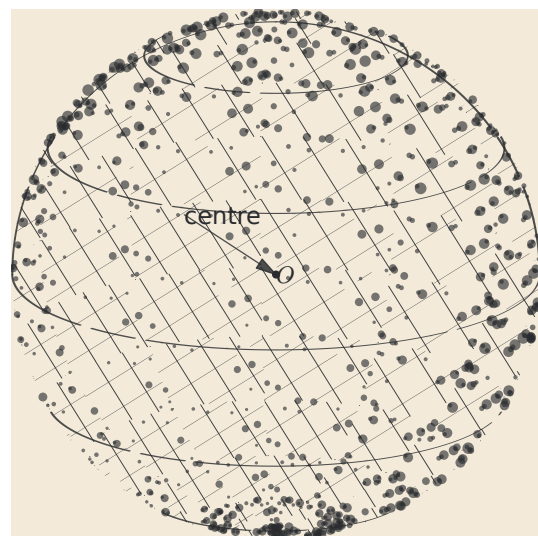
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let P = orthographic(6, 3, 2)
#let b = sphere((0, 0, 0.9), 0.9)

#pencil-picture(width: 7cm, projection: P,
// Les hachures croisées sont le ton principal ;
le stippling est optionnel.
  draw(surface(b), stipple: none),
  draw(b, m: 7,
    frontpen: pencil-pen(paint: pencil-ink,
thickness: 0.7pt,
    roughness: 0.18, pressure: 0.12, passes: 1,
seed: 7)),
  point3([$0$], (0, 0, 0.9), pen: pencil-ink),
  callout3[centre], (0, 0, 0.9), (1.3, 0.3, 1.5),
pen: pencil-ink),
)
```

Résultat vectoriel



pencil-pen(...) est utile pour contrôler une arête particulière ; picture(style: "pencil") est préférable pour une scène complète, car il traite aussi les contours de surfaces et les meshpen tout en retirant les lignes cachées implicites. Un backpen explicite permet toutefois de les demander. vintage-stipple(...) peut être passé à draw(surface(...), stipple: ...) ; le style pencil le recolorie en graphite tout en conservant les points

séparés du contour. `max-patch-count`: 96 limite par défaut la densité du fallback pur sur chaque patch projeté ; augmentez cette valeur pour une trame plus dense. Pour désactiver les textures d'une surface, passez explicitement `hatch: none` et `stipple: none`.

Contour normal et remplissage gravé

Pour obtenir le motif de la planche source `examples/shading-atlas.typ` avec un contour vectoriel classique, `vintage-shaded` compose une surface hachurée et un second tracé au premier plan. Les marques restent posées sur la géométrie avant projection ; le contour peut rester normal avec `plain: true`.

Contour normal et remplissage gravé

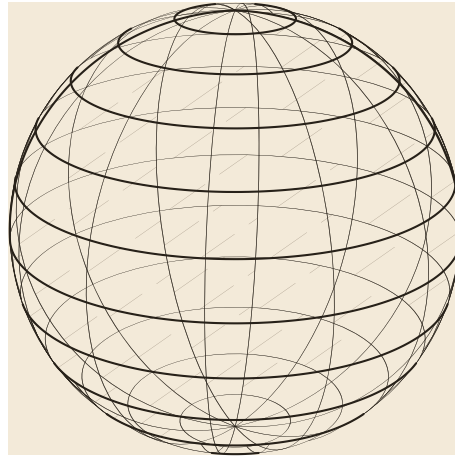
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let shape = sphere((0, 0, 0), 1)
#let hatch = vintage-hatch(
  count: 760,
  length: 0.22,
  crosshatch: 0.76,
  seed: 11,
)

#vintage-picture(width: 6cm, projection:
  perspective(5.6, 4.2, 2.8),
  vintage-shaded(
    shape,
    hatch: hatch,
    outline: (plain: true, paint: vintage-ink,
    thickness: 0.78pt),
  ),
)
```

Résultat vectoriel



`outline: auto` reprend la plume elliptique du shading ; `outline: none` supprime le contour. Pour un contour gravé comme les silhouettes de l'atlas, remplacez `plain: true` par une `pencil-pen(...)`.

Sketch 2D statique et projection 3D

La fonction `sketch` reprend un vocabulaire simple inspiré de `p5.js`, mais le callback est évalué une seule fois par Typst. `sketch-random` et `sketch-noise` sont déterministes ; les primitives sont émises dans un SVG vectoriel. Le contexte possède aussi `project3`, ce qui permet de transformer une courbe 3D en chemin 2D sans quitter le document statique.

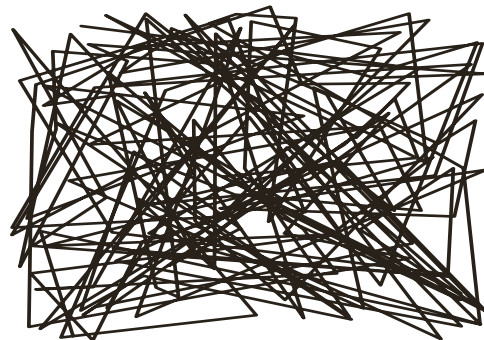
Sketch statique déterministe

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let scene = ctx => {
  let points = ()
  for i in range(160) {
    points.push((
      (ctx.random)(index: i * 2, a: 5, b: 95),
      (ctx.random)(index: i * 2 + 1, a: 5, b: 95),
    ))
  }
  ((ctx.polyline)(points, stroke: vintage-ink,
  width: .7),)
}
#sketch(width: 7cm, height: 5cm, seed: 17, draw:
scene)
```

Résultat vectoriel



`mesh-edges(vertices, faces)` fournit les chemins d'un wireframe dédoublé ; utilisez `boundary: true` pour ne garder que les arêtes de bord, puis `draw(mesh-edges(...), pen: ...)` dans une `picture 3D`. Pour un solide de révolution complet, `solid-plot` (alias `plot3d`) compose directement la surface, les hachures, le wireframe et le contour dans la même projection.

Surfaces paramétriques et maillages

Grille paramétrique

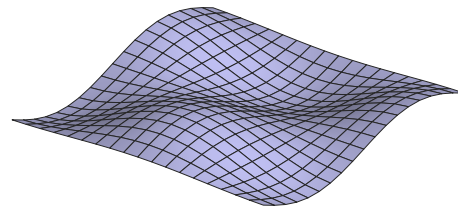
Grille paramétrique colorée avec maillage

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let vague = surface-grid(
  (u, v) => (u, v, 0.2 * calc.sin(2 * u) *
    calc.cos(2 * v)),
  -1.5, 1.5, -1.5, 1.5, nu: 18, nv: 18,
)
#picture(size: 6cm, projection: orthographic(5, 4,
3),
  draw(vague, withopacity(paleblue, 0.85), meshpen:
pen(black, 0.2pt)),
)
```

Résultat vectoriel



Le matériau et le `meshpen` doivent être passés au **même** `draw`. Un second `draw(vague, meshpen: black)` redessinerait la surface avec son matériau par défaut, noir, et pourrait masquer la couleur de la première passe.

Signature :

```
surface-grid(f, u0, u1, v0, v1,
  nu: 16, nv: 12,
  cyclic-u: false, cyclic-v: false,
  normal: auto, color: none)
```

`f(u,v)` retourne un triplet. Les normales sont moyennées aux sommets. On peut fournir `normal(u,v)` pour un ombrage analytique et `color(u,v)` pour une couleur par cellule. `parametric-surface` et `parametric` sont des alias.

Champ de hauteurs

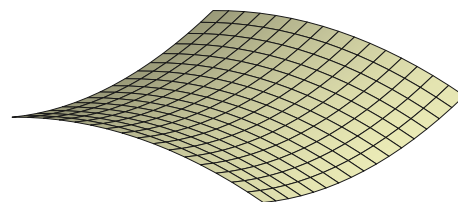
Champ de hauteurs

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let selle = heightfield(
  (x, y) => 0.15 * (x * x - y * y),
  -1, 1, -1, 1, nx: 16, ny: 16,
)
#picture(width: 6cm, projection: orthographic(5, 4,
3),
  draw(selle, withopacity(paleyellow, 0.9), meshpen:
pen(black, 0.2pt)),
)
```

Résultat vectoriel



Maillage indexé

Maillage lisse et maillage facetté

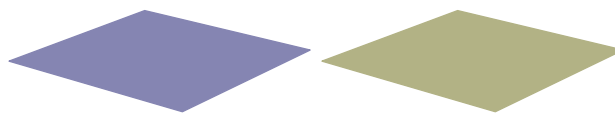
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let vertices = (
  (0, 0, 0), (1, 0, 0), (1, 1, 0), (0, 1, 0),
)
#let faces = ((0, 1, 2, 3),)
#let carré = surface-mesh(vertices, faces)
#let facetté = surface-mesh(vertices, faces, smooth:
false)

#grid(columns: 2, gutter: 6pt,
  picture(width: 4cm, draw(carré, paleblue)),
  picture(width: 4cm, draw(facetté, paleyellow)),
)
```

Résultat vectoriel



Les indices sont zéro-based par défaut. indices: "one" accepte les indices commençant à 1. smooth: false conserve une normale constante par face, utile pour les polyèdres.

Figures pédagogiques collège

Le module d'annotations est chargé automatiquement par l'import étoilé.

Sommets et flèches de rappel

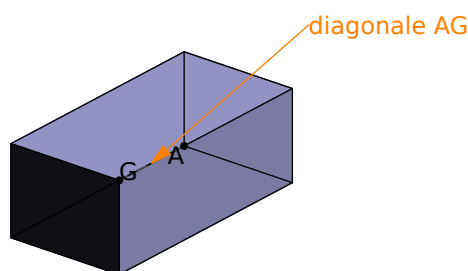
Sommet, rappel et diagonale effectivement tracée

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let A = (0, 0, 0)
#let G = (4, 2, 1.5)
#picture(size: 6cm, projection: orthographic(5, 4,
3),
  draw(cuboid(length: 4, width: 2, height: 1.5),
paleblue),
  draw(box3(A, G), black),
  draw(line3(A, G), pen(gray(0.35), dashed)),
  point3([A], A, align: SW), point3([G], G, align:
NE),
  callout3([diagonale AG], interp(A, G, 0.5),
(-1, 1.5, 2), pen: orange),
)
```

Résultat vectoriel



callout3 ne dessine pas l'objet désigné. Il dessine uniquement la ligne de rappel et la flèche. Le segment, la diagonale ou la hauteur doit être ajouté séparément avec draw(line3(...)) ou construction3(...).

Lignes de cote

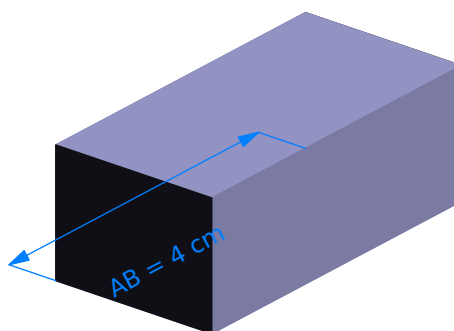
Cote avec lignes d'attache automatiques

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#picture(size: 6cm, projection: orthographic(5, 4,
3),
  draw(cuboid(length: 4, width: 2, height: 1.5),
paleblue),
  dimension3([AB = 4 cm], (0, 0, 0), (4, 0, 0),
offset: (0, -0.6, 0), align: S, pen: royalblue),
)
```

Résultat vectoriel



offset déplace la ligne de cote en 3D et crée automatiquement les deux lignes d'attache. Passer extensions: false lorsque la ligne de cote et ses flèches suffisent, sans petits traits supplémentaires. pen règle la ligne et les

flèches de la cote ; solid, dashed et dotted sont disponibles. extension-pen règle séparément les lignes d'attache qui restent visibles. Le texte de dimension3 suit la tangente projetée de la ligne de cote, y compris pour une cote oblique, et est retourné modulo 180° pour rester lisible. Utiliser rotate: false ou un angle explicite dans dimension3 pour un contrôle manuel. L'annotation est centrée sur la ligne et dx/dy permettent un réglage fin dans le plan projeté sans déplacer la cote 3D. Les mêmes paramètres ajustent les noms avec point3. right-angle3(a,b,c) place une marque d'angle droit au point b pour l'angle a-b-c.

Codages d'arêtes et d'angles

Les conventions de manuels peuvent être composées directement dans la même picture :

```
#let v = cuboid-vertices(length: 4, width: 2, height: 1.5)
#picture(width: 7cm, projection: orthographic(5, 4, 3),
  draw(cuboid(length: 4, width: 2, height: 1.5), withopacity(paleblue, .6)),
  cuboid-edges(v, visible: black, hidden: pen(gray(.45), .45pt, dashed)),
  point3($A$, v.A), point3($A'$, v.E), point3($O$, (2, 1, .75)),
  equal-edges3(((v.A, v.B), (v.D, v.C)), count: 1, pen: royalblue),
  angle-code3(v.B, v.A, v.E, right: true, pen: red),
  dimension3([AB = 4 cm], v.A, v.B, offset: (0, -.6, 0), pen: royalblue),
)
```

edge-code3 dessine un ou plusieurs petits traits perpendiculaires à une arête ; equal-edges3 réutilise le même nombre de traits sur une liste de couples de sommets. angle-mark3 dessine un arc pour un angle non droit, tandis que angle-code3(..., right: true) choisit le carré d'angle droit. Les labels de point3, callout3, dimension3 et des marqueurs peuvent être du contenu mathématique, par exemple \$A\$, \$O\$ ou \$A'\$.

La planche complète est dans examples/college.typ; le catalogue directement copiable examples/college-annotations.typ couvre le pavé droit, le prisme triangulaire, la pyramide, le cylindre, la sphère et le repère, avec arêtes cachées, égalités, angles, hauteurs, rayons, génératrices et cotes.

RTL et textes multilingues

Le moteur de dessin est compatible avec les labels arabes et hébreux. La langue et la direction doivent être fixées dans le document :

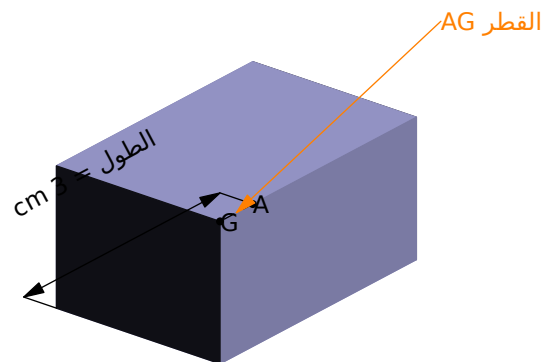
RTL : textes arabes et géométrie latine

Code Typst

```
#import "@preview/solids3d:0.4.0": *
#set text(font: "DejaVu Sans", lang: "ar", dir: rtl)

#let v = cuboid-vertices(length: 3, width: 2,
  height: 1.5)
#picture(width: 7cm, projection: orthographic(5, 4,
  3),
  draw(cuboid(length: 3, width: 2, height: 1.5),
    paleblue),
  diagonal3(v.A, v.G),
  point3([A], v.A), point3([G], v.G),
  dimension3([3 = الطول cm], v.A, v.B,
    offset: (0, -.4, 0)),
  callout3([القطر AG], interp(v.A, v.G, 0.5),
    (-1, 1.5, 2), pen: orange),
)
```

Résultat vectoriel



Les mots sont traduits, mais les identifiants mathématiques, lettres de sommets, nombres, symboles et unités (A, AG, cm, =, 3) restent latins. La direction du texte ne miroir pas la géométrie : E, W, N et S restent des directions physiques. Pour une expression explicitement LTR dans un paragraphe RTL :

```
#text(dir: ltr)[AB = 4 cm]
```

L'exemple compilable est examples/rtl.typ.

Nouveautés 0.2 : nibart, gravure, nœuds, accolades et planches

La version 0.2 intègre nibart (@preview/nibart:0.3.0, licence MIT), une bibliothèque de plumes calligraphiques et de chemins de type MetaPost écrite en Typst pur. Elle apporte cinq choses : le moteur de plume de picture, le style "engraving", les nœuds knot3, les accolades brace3 et le cartouche plate.

Dans ce chapitre chaque exemple affiche son code à gauche et le rendu **obtenu en évaluant ce même code** à droite : il n'y a pas de copie qui puisse diverger. Les rendus ci-dessous utilisent tous la même lumière `light((-1, 0.9, 1.1))`, qui vient de la gauche et d'en haut.

Moteur de plume : engine

`picture(..., engine: "nibart")` (valeur par défaut) calcule les traits à plume et les contours avec nibart ; c'est le seul moteur depuis la 0.4 (`engine: "wasm"` et le module WebAssembly ont été supprimés).

Le style engraving : la gravure au burin

`picture(style: "engraving", engraving: engraving(...), ...)` remplace les surfaces ombrées par une **trame de lignes parallèles** dont l'épaisseur suit le ton de la surface, comme sur un billet de banque ou une gravure sur cuivre : lignes épaisses dans l'ombre, fines dans la lumière, interrompues dans les reflets ; une seconde famille croisée apparaît dans les ombres profondes. Les contours sont tracés à la plume plate de nibart (pleins et déliés). Les lignes cachées sont éliminées par l'algorithme du peintre : une face proche coupe les lignes qui avaient été posées derrière elle.

Gravure droite ou ondulée (guilloché)

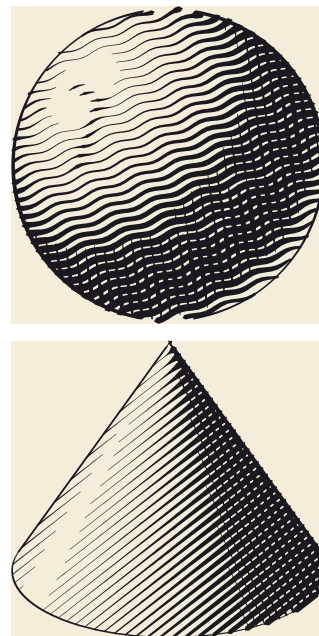
Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let L = light((-1, 0.9, 1.1))
#let P = orthographic(5, 4, 3)
#let billet = engraving(
  wave: (amplitude: 0.45pt, length: 11pt, shift:
0.35),
  angle: 20, spacing: 2.4pt)

#picture(size: 4.2cm, projection: P, style:
"engraving",
  engraving: billet, light: L,
  engraved(sphere((0, 0, 0), 1)))
#picture(size: 4.2cm, projection: P, style:
"engraving", light: L,
  engraved(cone((0, 0, 0), 1, 1.6, Z, 1),
    edges: linewidth(0.4)))
```

Résultat vectoriel



Paramètres de `engraving(...)`.

Paramètre	Défaut	Rôle
spacing	2.1pt	distance entre deux lignes de la trame
angle	38	direction (en degrés) de la première famille
cross	true	ajoute la famille croisée dans les ombres profondes
cross-angle	78	écart (degrés) entre la famille croisée et angle
wave	none	none (lignes droites) ou (amplitude: 0.5pt, length: 14pt, shift: 0.0) : ondulation de type billet de banque ; shift décale la phase d'une ligne à l'autre (moiré de guilloché). Seule la première famille ondule, la famille croisée reste droite
gain	1.0	quantité d'encre : > 1 plus sombre, < 1 plus clair
minimum	0.13pt	trait le plus fin affiché ; en dessous, la ligne est omise
contours	true	trace les contours à la plume plate de nibart
nib-width	1.35	largeur de la plume plate, multiple de l'épaisseur du trait
nib-angle	38	inclinaison de la plume plate, en degrés
nib-ratio	0.36	rapport épaisseur du délié / plein

ink, paper	auto	couleurs de l'encre et du papier (engraving-ink, engraving-paper)
step	1.6pt	pas d'échantillonnage le long d'une ligne

Aide `engraved(shape, m: 48, rim: auto, edges: none, color: none)`. Elle combine en un appel la surface hachurée (`draw(surface(shape))`), la silhouette (`m` échantillons ; `rim` = plume du contour, `none` pour l'omettre) et, si `edges` est une plume, les arêtes du solide. Pour un tore standard (axe Z), `silhouette` calcule le contour exact — branche extérieure et bord du trou — et supprime les parties cachées derrière le tore lui-même, en orthographie comme en perspective. Les matières de l'éclairage du mode gravure n'ont pas de reflet spéculaire, qui rendrait la tache lumineuse irrégulière.

Nœuds et entrelacs : `knot3`

`knot3(..brins, samples: 120, width: 7pt, colors: auto, style: "gap", gap: auto, outline: auto, ink: auto, flips: ())` dessine des courbes fermées de l'espace comme des **tubes ombrés** (contour, côté ombre, corps, reflet — quatre traits de plume nibart). Les croisements sont détectés après projection ; le brin qui passe dessous est celui dont la profondeur est la plus grande. Un brin est une fonction `t => (x, y, z)` sur `[0,1[` ou un tableau de points.

Interruption ("gap"), tissage ("weave") et gravure

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let L = light((-1, 0.9, 1))
#let P = orthographic(3, 2, 4)
#picture(size: 4cm, projection: P, light: L,
  knot3(trefoil(), width: 10pt))
#picture(size: 4cm, projection: P, light: L,
  knot3(trefoil(), width: 10pt, style: "weave",
    colors: (rgb("#2e7d5b"))))
#picture(size: 4cm, projection: P, style:
  "engraving", light: L,
  knot3(figure-eight(), width: 9pt, style: "weave"))
```

Résultat vectoriel



Paramètre	Défaut	Rôle
<code>..brins</code>	—	courbes <code>t => (x, y, z)</code> ou tableaux de points ; on peut passer directement le tableau renvoyé par <code>hopf-link()</code> ou <code>borromean()</code> avec ..
<code>samples</code>	120	points par brin (augmenter pour les nœuds à nombreux croisements)
<code>width</code>	7pt	diamètre du tube sur le papier
<code>colors</code>	auto	une couleur par brin (cycle) ; ignoré en mode <code>engraving</code> , où le tube est de la couleur du papier
<code>style</code>	"gap"	"gap" : le brin dessous est interrompu ; "weave" : tubes continus, le brin dessus est redessiné sur le croisement
<code>gap</code>	auto	jeu laissé de part et d'autre du brin dessus en mode "gap" (longueur ; auto = 30 % de width)

outline	auto	épaisseur du contour (auto = 14 % de width, 0pt : sans contour)
ink	auto	couleur du contour et du côté ombre en mode engraving
flips	()	indices des croisements dont on inverse dessus/dessous (le tableau (0, 1, 2) donne l'image miroir d'un trèfle)

Courbes fournies. trefoil(scale: 1.0) (trèfle 3_1), figure-eight(scale: 1.0) (nœud de huit 4_1), torus-knot(p, q, R: 2.0, r: 0.9) (nœud torique $T(p, q)$ sur un tore de rayons R et r), hopf-link(R: 1.0) (deux cercles liés) et borromean(a: 2.0, b: 1.2) (trois ellipses : aucune paire n'est liée, mais les trois sont inséparables).

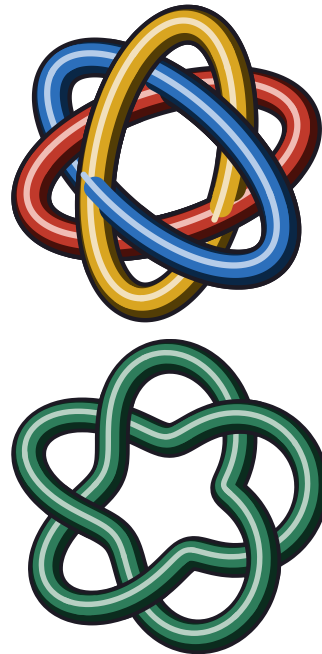
Anneaux de Borromée et nœud torique $T(2,5)$

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let L = light((-1, 0.9, 1))
#picture(size: 4.2cm, projection: orthographic(1, 1, 1),
  light: L,
  knot3(.borromean(a: 2, b: 1.25), width: 7pt,
    style: "weave"))
#picture(size: 4.2cm, projection: orthographic(1, 1, 5),
  light: L,
  knot3(torus-knot(2, 5, R: 2, r: 0.8), width: 8pt,
    samples: 160, style: "weave",
    colors: (rgb("#2e7d5b"),)))
```

Résultat vectoriel



Les nœuds sont dessinés comme une couche, dans l'ordre des commandes de picture : ils ne sont pas triés en profondeur par rapport aux solides qu'ils doivent recouvrir. Placez-les après les solides qu'ils doivent recouvrir. Le style "gap" retire de grandes portions de tube lorsque deux croisements sont proches (anneaux de Borromée) : utiliser alors "weave" ou un width plus fin.

Accolades calligraphiques : brace3

brace3(a, b, label: none, offset: (0, 0, 0), kind: "brace", amplitude: 9pt, flip: false, light: 0.35pt, heavy: 1.7pt, pen: auto, gap: 5pt) joint deux points de l'espace par une accolade (kind: "paren" ou "paren-straight" pour une parenthèse) dont la plume enfle dans les courbures (delimiter de nibart). Les points sont d'abord décalés de offset en 3D, puis projetés. flip choisit l'autre côté (par défaut, l'accolade se creuse à gauche du sens de a vers b sur la figure). amplitude est la hauteur de la pointe sur le papier, light et heavy sont les largeurs de plume aux extrémités et aux renflements, label est centré au-delà de la pointe à la distance gap.

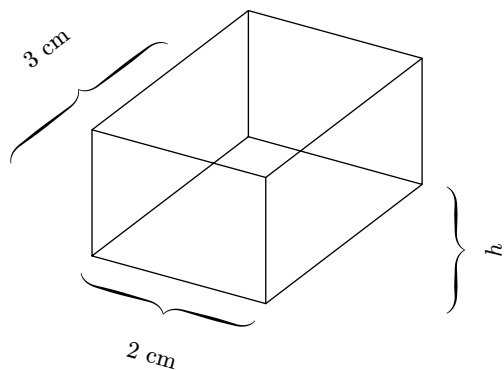
Cotes d'un pavé droit

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#picture(size: 6.5cm, projection: orthographic(5, 3,
3),
light: light((-1, 1, 1)),
draw(box3((0, 0, 0), (3, 2, 1.4)), black),
brace3((0, 0, 0), (3, 0, 0), offset: (0, -0.15,
0),
label: $3 "cm"$, flip: true),
brace3((3, 0, 0), (3, 2, 0), offset: (0.15, 0, 0),
label: $2 "cm"$, flip: true),
brace3((3, 2, 0), (3, 2, 1.4), offset: (0.15,
0.15, 0),
label: $h$))
```

Résultat vectoriel



Cartouche gravé : plate

plate(body, title: none, subtitle: none, number: none, caption: none, ink, paper, pad: 16pt, band: 2.2pt, corner: 9pt, border: true, width: auto, font: auto, ornament: true) entoure une figure d'un cadre à coins échancrés (double filet, plume plate) et d'un fleuron calligraphique sous le titre. number produit « Pl. I », caption la légende en italique. width accepte une longueur ou une proportion (100%).

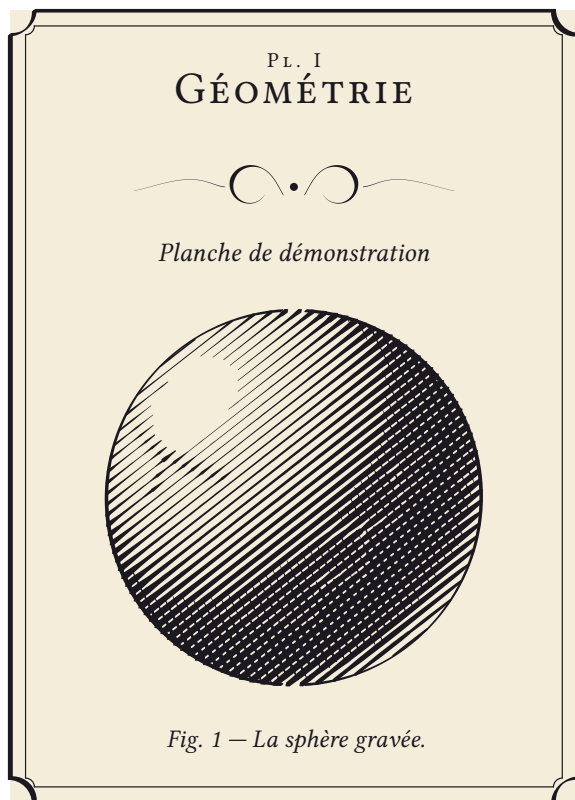
Planche

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#plate(title: "Géométrie", subtitle: "Planche de
démonstration",
number: "I", caption: [Fig. 1 – La sphère
gravée.],
width: 7.5cm,
picture(size: 5cm, projection: orthographic(4, -3,
2.5),
style: "engraving", light: light((-1, 0.9,
1.1)),
engraved(sphere((0, 0, 0), 1)))
```

Résultat vectoriel



L'exemple `examples/planche.typ` assemble neuf figures dans un cartouche A4 (sphère, tore, cône, cylindre, trèfle, anneaux de Borromée, nœud $T(2,5)$, nœud de huit et entrelacs de Hopf) ; `examples/engraving.typ` et `examples/knots.typ` montrent des variantes.

Correspondance avec Asymptote

Asymptote	Typst solids
size(6cm)	picture(size: 6cm, ...)

currentprojection = orthographic(...)	projection: orthographic(...)
perspective(...)	projection: perspective(...)
triple	triplet (x, y, z)
A--B--C	line3(A, B, C)
A..B..C	spline3(A, B, C)
surface(b)	surface(b) OU surface(revolution)
draw(surface(b), pen)	draw(surface(b), pen)
draw(b, m, n, ...)	draw(b, m: ..., n: ..., ...)
currentlight = White	picture(light: White, ...)
opacity(.5)	withopacity(color, 0.5)
label(...), dot(...)	label(...), dot(...), point3(...)
intersectionpoints	non implémenté

Les noms et la structure restent volontairement proches, mais Typst utilise les arguments nommés avec : et les transformations avec apply.

Commandes et paramètres — code à gauche, rendu à droite

Cette section rassemble les commandes les plus utilisées sous forme de recettes complètes. Dans chaque encadré, le code et les paramètres sont à gauche ; le rendu vectoriel correspondant est à droite. Les valeurs non mentionnées reprennent les valeurs par défaut de l'API.

Construire une figure : picture, projection et draw

Commande	Paramètres principaux
picture(...)	size, width, height, projection, backend, shading, zsort, refine, style, light
orthographic(...)	camera, up, target, zoom ; projection parallèle
perspective(...)	camera, up, target, zoom ; profondeur perspective
draw(...)	pen, frontpen, backpen, longitudinalpen, m, n, hatch, stipple
surface(...)	n pour une révolution ; conversion d'un solide ou d'un maillage en surface

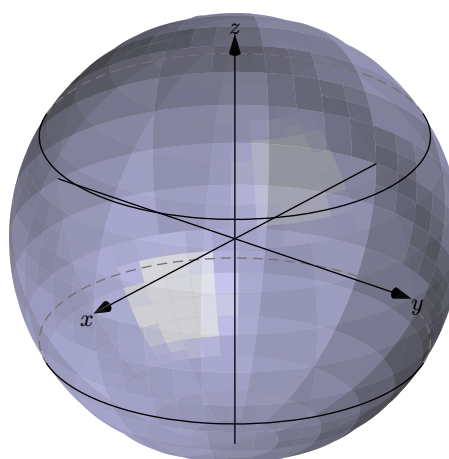
Projection, surface, squelette et axes

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let P = orthographic(5, 4, 3, zoom: 1.05)
#let b = sphere(0, 1, n: 16)
#picture(
  size: 6cm,
  projection: P,
  shading: "smooth",
  backend: "typst",
  draw(surface(b), withopacity(paleblue, 0.55)),
  draw(b, m: 4,
    frontpen: black,
    backpen: pen(gray(.45), linetype("7 5")),
    longitudinalpen: nullpen),
  xaxis3($x$, Arrow3), yaxis3($y$, Arrow3),
  zaxis3($z$, Arrow3),
)
```

Résultat vectoriel



Chemins et courbes 3D

Commande	Paramètres principaux
line3(..points, cyclic: false)	Chemin polygonal ; cyclic: true ferme le chemin
spline3(..points, cyclic: false)	Chemin de Bézier 3D avec connecteurs de Hobby

guide3(..points, join: ...)	join: "--", "..", tensions et contrôles
graph3(f, a, b, ...)	n, join ; f(t) retourne (x, y, z)
Arc3(c, v1, v2, ...)	normal, direction, n ; arc cubique approché

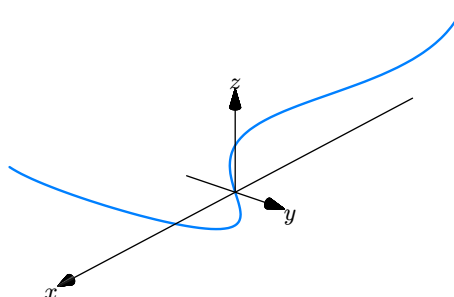
Courbe paramétrique et connecteurs de Bézier

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let g = graph3(
  t => (t, .35 * calc.sin(3 * t), .25 * t * t),
  -1.6, 1.6,
  n: 48,
  join: "..",
)
#picture(size: 6cm, projection: orthographic(5, 4, 3),
  draw(g, pen(royalblue, 0.9pt)),
  xaxis3($x$, Arrow3), yaxis3($y$, Arrow3),
  zaxis3($z$, Arrow3),
)
```

Résultat vectoriel



Solides, surfaces paramétriques et tubes

Commande	Paramètres principaux
sphere(c, r, n: 16)	c, r, n ; solide de révolution sphérique
cylinder(c, r, h, axis)	c, r, h, axis, n
cone(c, r, h, axis, ratio)	ratio, n, angle de révolution éventuel
revolution(g, axis, ...)	angle1, angle2 ; profil g et axe
surface-grid(f, u0, u1, v0, v1, ...)	nu, nv, cyclic-u, cyclic-v, normal, color
surface-mesh(vertices, faces, ...)	indices, normal, smooth, color
tube3(spine, ...)	width, n, samples, caps, color ; width est le diamètre

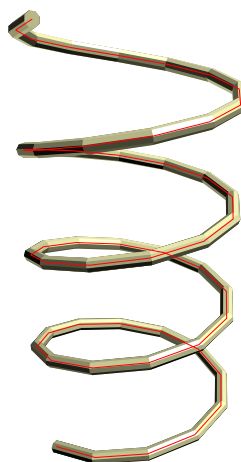
Tube polygonal autour d'une hélice

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let tau = 2 * calc.pi
#let helix = graph3(
  t => (1.0 * calc.cos(t * 1rad),
    1.0 * calc.sin(t * 1rad), -1 + t / tau),
  0, 4 * tau, n: 48, join: "--",
)
#let wire = tube3(
  helix,
  width: 0.12,
  n: 8,
  samples: 1,
  caps: true,
  color: paleyellow,
)
#picture(size: 6cm, projection: perspective(4, 5, 3),
  draw(surface(wire)),
  draw(helix, pen(red, 0.3pt)),
)
```

Résultat vectoriel



Annotations pédagogiques

Commande	Paramètres principaux
<code>point3(label, p, ...)</code>	<code>align</code> , <code>pen</code> , <code>size</code> , <code>dx</code> , <code>dy</code>
<code>callout3(text, target, label-at, ...)</code>	<code>align</code> , <code>pen</code> , <code>arrow</code> , <code>dx</code> , <code>dy</code>
<code>dimension3(text, a, b, ...)</code>	<code>offset</code> , <code>align</code> , <code>pen</code> , <code>arrow</code> , <code>rotate</code> , <code>dx</code> , <code>dy</code> , <code>extensions</code> , <code>extension-pen</code>
<code>angle-code3(a, b, c, ...)</code>	<code>right</code> , <code>size</code> , <code>radius</code> , <code>pen</code> , <code>label</code> , <code>align</code>
<code>right-angle3(a, b, c, ...)</code>	<code>size</code> , <code>pen</code> ; carré géométrique
<code>angle-mark3(a, b, c, ...)</code>	<code>radius</code> , <code>pen</code> , <code>label</code> , <code>align</code> ; arc non droit
<code>edge-code3(a, b, ...)</code>	<code>count</code> , <code>size</code> , <code>spacing</code> , <code>at</code> , <code>direction</code> , <code>pen</code>

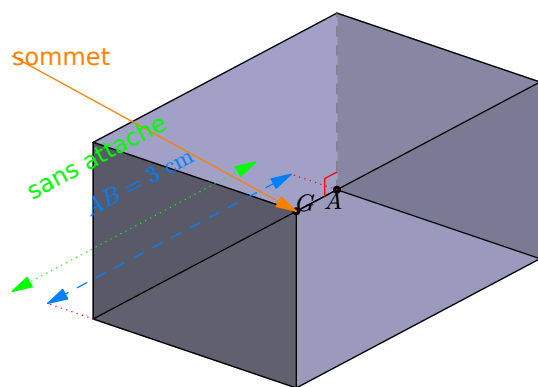
Cotes, attaches, styles et angle droit

Code Typst

```
#import "@preview/solids3d:0.4.0": *

#let v = cuboid-vertices(length: 3, width: 2,
height: 1.5)
#picture(size: 7cm, projection: orthographic(5, 4,
3),
draw(cuboid(length: 3, width: 2, height: 1.5),
withopacity(paleblue, 0.55)),
cuboid-edges(v, visible: black,
hidden: pen(gray(.45), .45pt, dashed)),
point3($A$, v.A, align: SW, dx: 2pt, dy: -1pt),
point3($G$, v.G, align: NE),
dimension3($A B = 3$ "cm"$, v.A, v.B,
offset: (0, -0.45, 0),
pen: pen(royalblue, dashed),
extension-pen: pen(red, dotted)),
dimension3([sans attache], v.A, v.B,
offset: (0, -0.8, 0),
pen: pen(green, dotted),
extensions: false),
angle-code3(v.B, v.A, v.E, right: true, pen: red),
callout3([sommet], v.G, (4, 0, 2.6), pen: orange),
)
```

Résultat vectoriel



Référence API condensée

Projections

#name — Projection parallèle. Arguments : caméra, up, target, zoom. **#name** — Projection perspective. Arguments : caméra, up, target, zoom. **#name** — Projette un triplet, un chemin ou un tableau. **#name** — Projette un point en paire 2D. **#name** — Projette un chemin 3D en chemin 2D.

Chemins

#name — Construit un chemin polygonal droit. **#name** — Construit une spline 3D de Hobby. **#name** — Construit un chemin avec connecteurs et contrôles. **#name** — Cercle cubique avec nombre de segments réglable. **#name** — Arc avec normale et direction. **#name** — Graphe d'une fonction réelle à valeurs 3D. **#name** — Extrait une portion temporelle d'un chemin. **#name** — Mesure et paramétrage par longueur.

Solides et surfaces

#name — Crée une surface de révolution différée. **#name** — Constructeurs de solides courants. **#name** — Convertit un chemin, une révolution, un patch ou une union en surface. **#name** — Crée un patch plan avec normale, centre, couleur et trous. **#name** — Extrude un chemin selon un vecteur. **#name** — Relie une base à un sommet. **#name** — Grille paramétrique avec normales de sommets. **#name** — Surface $z = f(x,y)$. **#name** — Maillage indexé avec normales moyennées ou facettées.

Rendu

#name — Projette, trie, éclaire, met à l'échelle et émet la scène. **#name** — Crée une commande de chemin, surface, révolution ou objet. **#name** — Remplit un chemin plan sans éclairage. **#name** — Squelette visible et caché d'une révolution. **#name** — Silhouette échantillonnée d'une révolution. **#name** — Points et contenu typographique projeté. **#name** — Axes avec labels et flèches. **#name** — Plumes vectorielles à passes irrégulières, déterministes et imprimables. **#name** — Raccourci graphite pour chemins et surfaces ; les backpens implicites des squelettes sont retirés. **#name** — Hachures et stippling par patch, plus matériau sépia optionnels. **#name** — Raccourcis de `picture(style: "vintage" | "pencil")`.

Éducation

#name — Marque un sommet ou un centre. **#name** — Flèche de rappel vers un élément déjà dessiné. **#name** — Ligne de cote avec flèches ; extensions: `false` supprime les lignes d'attache et `extension-pen` règle leur style indépendamment. **#name** — Marque d'angle droit en 3D. **#name** — Ligne de construction en pointillés. **#name** — Pavé droit fermé. **#name** — Pyramide régulière à base polygonale. **#name** — Prisme régulier droit. **#name** — Diagonale ou hauteur réellement tracée. **#name** — Base polygonale réellement composée de n sommets. **#name** — Polyèdres réguliers supplémentaires. **#name** — Tronc de cône et prisme droit triangulaire. **#name** — Sommets A-H et arêtes réutilisables d'un pavé.

Façades de modules Asymptote

Les fichiers de façade regroupent les fonctions sans copier le code source d'Asymptote. L'entrée principale les exporte aussi sous forme de modules :

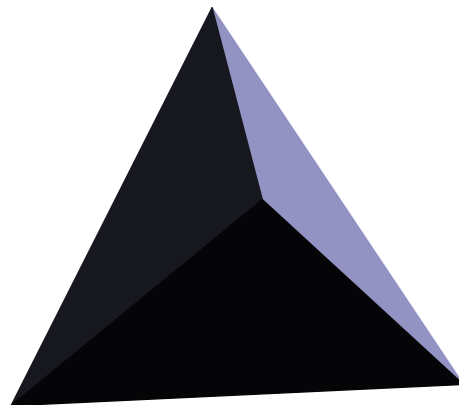
Façades groupées inspirées d'Asymptote

Code Typst

```
#import "@preview/solids3d:0.4.0": three,
three_surface, three_light, graph3mod, college

#let P = three.orthographic(5, 4, 3)
#let T = college.regular-tetrahedron(radius: 1)
#three.picture(
  size: 6cm,
  projection: P,
  three.draw(T, three.paleblue),
)
```

Résultat vectoriel



Correspondance des façades :

- `three` : vecteurs, chemins, projection, rendu et flèches ;
- `three_surface` : surfaces, maillages et solides ;
- `three_light` : couleurs, pens, matériaux et lumières ;
- `graph3mod` : courbes `graph3-plot` et `parametric-plot3` ;
- `college` : annotations et polyèdres scolaires ;
- `solidsmod` : révolution, squelette et silhouette ;
- `obj3` : lecture OBJ.

Les fonctions les plus courantes sont également exportées directement par `@preview/solids3d:0.4.0`, ce qui permet l'import étoilé des exemples.

Performance et qualité

Les ordres de grandeur dépendent du nombre de patches et de la taille de la figure :

- `shading: "none"` est le mode le moins coûteux ;
- `shading: "flat"` est plus compact que `smooth` ;
- `n`, `nu`, `nv` contrôlent la finesse du maillage ;
- `refine: false` évite les découpes de silhouette et les subdivisions ;
- `backend: "typst"` évite les gradients SVG mais utilise des couleurs plates ;

- une surface paramétrique 16×12 crée 192 patches : augmenter `nu` et `nv` seulement si la silhouette le nécessite.

Le PDF reste vectoriel. Le backend SVG crée des gradients vectoriels intégrés au PDF ; il ne s'agit pas d'une capture raster.

Limites connues

Le package ne remplace pas le rendu OpenGL ou PRC d'Asymptote. Ne sont pas implémentés :

- `intersectionpoints(path3, surface)` ;
- `polyhedron_js` ;
- le rendu `settings.render` ;
- `render(merge=true)` ;
- les transformations 3D de `Label` ;
- un depth-buffer rasterisé pour les lignes arbitraires.

Pour les lignes cachées, utiliser les parties `.back` avant les surfaces et les parties `.front` après les surfaces. `zsort: "global"` trie les patches de surface de toutes les commandes, mais ne transforme pas la scène en moteur OpenGL.

Dépannage

Typst ne trouve pas le package

Vérifier l'installation et la version dans l'import :

```
./install.sh
```

```
#import "@preview/solids3d:0.4.0": *
```

Les textes arabes sont mal placés

Fixer `lang`, `dir` et une police contenant les glyphes. Pour une formule ou une unité, utiliser un contenu LTR local. Les alignements `E` et `w` sont physiques, pas logiques.

Une diagonale n'apparaît pas

Une annotation `callout3` n'ajoute pas la géométrie. Dessiner séparément :

```
draw(line3(A, G), dashed)
callout3([diagonale AG], interp(A, G, 0.5), (-1, 1, 2))
```

Les coutures des patches sont visibles

Utiliser `shading: "smooth"`, augmenter `n/nu/nv`, activer le backend SVG et conserver la valeur par défaut de `seam`. Pour un PDF très léger, choisir `shading: "flat"` en acceptant un ombrage moins lisse.

La figure est trop lente

Réduire `n`, `nu`, `nv`, désactiver `refine` et essayer `shading: "flat"` ou `"none"`. Éviter de créer une sphère de plusieurs milliers de patches dans une grille de nombreuses figures.

Organisation et livraison

```
solids3d/
├── lib.typ          API exportée
├── typst.toml       métadonnées et version
├── src/
│   ├── vec.typ      vecteurs et transformations
│   ├── path3.typ     chemins 3D et Bézier
│   ├── projection.typ orthographique et perspective
│   ├── pens.typ      couleurs, matériaux, lumières
│   ├── surface.typ   patches, surfaces, maillages
│   ├── solids.typ    révolutions, squelettes, silhouettes
│   ├── picture.typ   commandes et moteur de rendu
│   ├── easy.typ      API simple : fig, paint, cube, brick
│   ├── engrave.typ, engraved.typ style gravure
│   ├── knots.typ, braces.typ, plate.typ nœuds, accolades, planches
│   ├── obj.typ       lecteur Wavefront OBJ
│   ├── education.typ annotations et solides scolaires
│   └── college.typ   polyèdres, diagonales et hauteurs de collège
├── three.typ, three-surface.typ, three-light.typ, graph3.typ façades
├── college.typ, obj3.typ, solids.typ, vintage.typ      modules publics
└── docs/        manuel (EN/FR), ce guide, sources et scripts de construction
```

```
|— examples/          sources des démonstrations
|— tests/             tests de compilation (sorties temporaires)
|— API.md  AUDIT.md  CHANGELOG.md  README.md  LICENSE
|— make-editions.sh   construit les deux éditions (Universe et locale)
```

Construire les deux éditions :

`./make-editions.sh`

Le script lance les tests, compile les exemples, construit le manuel (anglais et français, chapitre par chapitre puis fusionné) et ce guide pour chaque édition, puis crée `solids3d-VERSION-universe.zip` (arborescence `packages/preview/solids3d/VERSION/`) et `solids3d-VERSION-local.zip` (installateurs `install.sh / install.ps1`, paquet sous `@local`), avec leurs sommes SHA-256.

Licence et références

Le code du package est distribué sous LGPL-3.0-or-later. Les algorithmes de solides sont un portage des idées et conventions des modules Asymptote indiqués dans le README. Les figures pédagogiques suivent les familles de solides documentées par [pas-cours](<https://ctan.org/pkg/pas-cours>), sans incorporer son code source.

Fin du manuel — solids3d 0.4.0